



Universidade do Minho

Licenciatura em Engenharia Informática

Fase 1

Projeto LI3

Airline Query Solver

PL6 – Grupo 63

Pedro Afonso Quesado Pacheco Pereira (a107332)

José Dinis Guimarães Machado

Octávio Pita Henriques

Novembro de 20

1. Introdução

O trabalho prático desenvolvido no âmbito da unidade curricular **Laboratórios de Informática III** tem como principal objetivo a aplicação dos conceitos de **modularização, encapsulamento e gestão eficiente de memória** na linguagem de programação **C**.

O projeto, denominado **Airline Query Solver**, tem como finalidade o **tratamento e análise de grandes volumes de dados** provenientes de múltiplos ficheiros **CSV**, contendo informação relativa a **aeroportos, voos, aeronaves, passageiros e reservas**. Esse tratamento envolve a implementação de um processo de **parsing**, no qual os dados são interpretados e transformados em **entidades representativas**, posteriormente armazenadas em **estruturas de dados específicas** designadas por *catálogos*.

Além do parsing, uma etapa crítica do projeto consiste na **validação rigorosa dos campos**, assegurando a integridade e a consistência dos dados processados. Os catálogos criados não só permitem o armazenamento eficiente da informação, como também servem de base à execução de **queries predefinidas**, formuladas a partir de requisitos específicos. Quando executadas, estas queries produzem resultados estruturados que são processados e exportados para ficheiros individuais de saída.

A complexidade do projeto é ampliada pelo facto de existir um **ficheiro de input** que contém, linha a linha, os comandos a executar. Cada comando define o tipo de query e os respetivos argumentos, exigindo uma **lógica adaptativa e flexível** que garanta o correto processamento das instruções e a geração dos resultados correspondentes.

O sistema foi concebido com uma **arquitetura modular**, em que cada componente desempenha uma função distinta — desde a leitura e validação de dados, à execução das queries e escrita dos resultados. Essa divisão clara de responsabilidades assegura a escalabilidade do código e a facilidade de manutenção.

Além disso, foi dada especial atenção à **gestão de memória**, com o uso intensivo do **Valgrind** para garantir a inexistência de *memory leaks*, e à definição de um **Makefile robusto e flexível**, que automatiza o processo de compilação e garante a correta ligação de todas as dependências.

Entre as características que se destacam, encontram-se a **abordagem sistemática de validação de dados**, o **uso da biblioteca GLib** para a implementação de estruturas de dados dinâmicas e a **integração automática de execução e comparação de resultados** através do executável **programa-principal**. Estas funcionalidades refletem a preocupação com a robustez, a clareza e o desempenho do sistema, assegurando uma base sólida para a segunda fase do projeto.

Em suma, esta primeira fase constituiu uma experiência essencial de consolidação de competências em engenharia de software, abrangendo desde a manipulação de ficheiros e parsing de dados até à gestão de estruturas complexas e execução dinâmica de consultas.

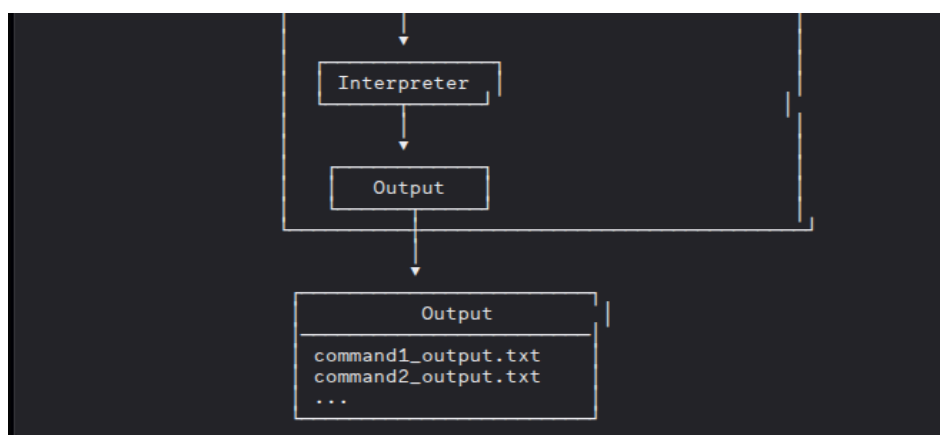
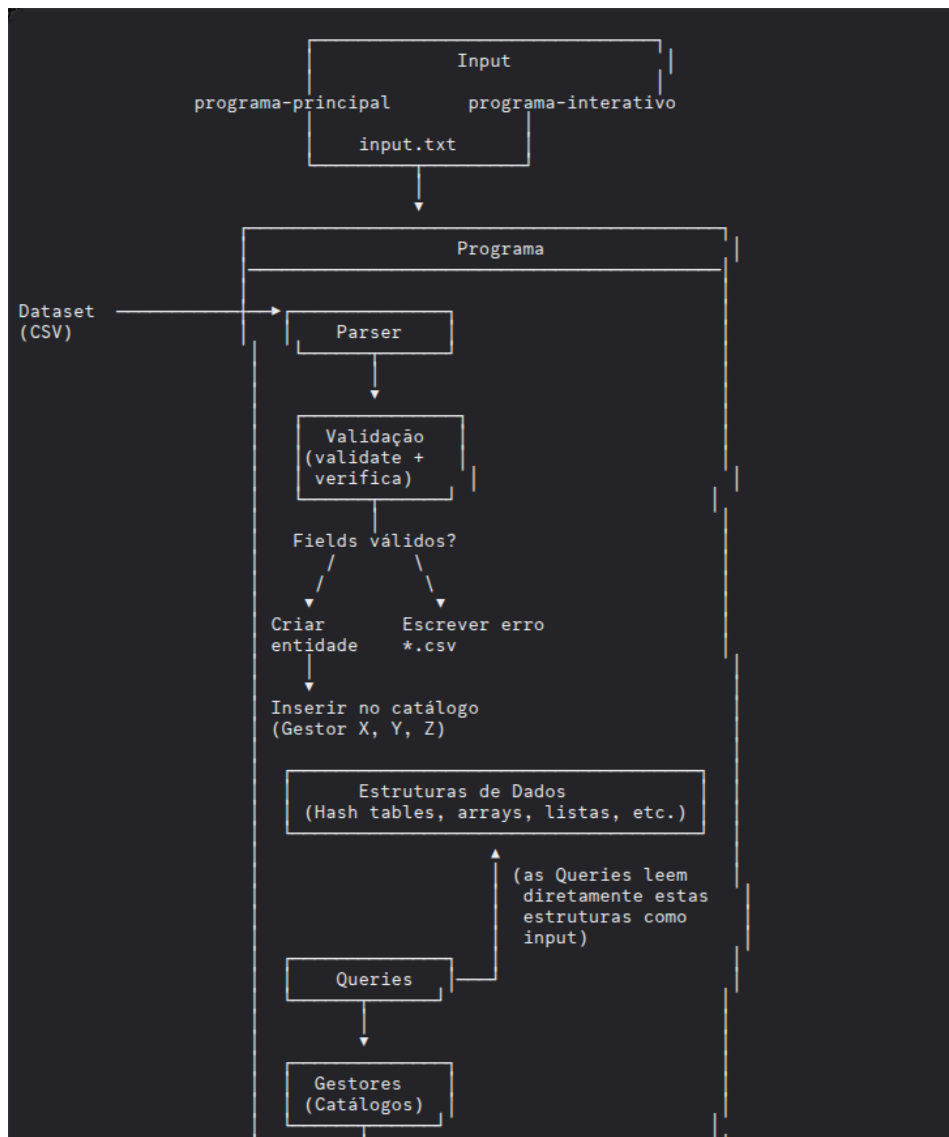
O desafio residiu não apenas na correta implementação de cada componente, mas também na sua **integração harmoniosa num sistema coeso, eficiente e totalmente funcional**.

2. Sistema

A arquitetura do sistema foi concebida com uma **abordagem modular**, assegurando uma estrutura organizada, extensível e de fácil manutenção. Cada módulo foi implementado com uma responsabilidade específica, comunicando com os restantes através de **interfaces bem definidas**, o que promove o **encapsulamento** e a **coerência estrutural** do código.

O projeto pode ser dividido em três grandes componentes lógicas:

1. **Processamento de Entrada** – responsável por carregar, interpretar e validar os dados provenientes dos ficheiros CSV e do ficheiro de input.
2. **Lógica de Negócio** – núcleo do sistema, que gere os catálogos e executa as queries.
3. **Geração de Resultados** – responsável pela formatação e escrita dos ficheiros de saída.



2.1 Estrutura Modular

O projeto encontra-se organizado em várias pastas, refletindo a divisão lógica e funcional dos módulos:

- **Executável (`programaPrincipal`)**

É o ponto de entrada do sistema. Recebe os caminhos dos datasets e do ficheiro de input, invoca o parser para carregar os dados e coordena a execução das queries. No final, chama o módulo de output para gerar os ficheiros `commandN_output.txt`.

- **Parser (`parser`)**

Responsável pela leitura dos ficheiros `.csv`. Cada linha é dividida em campos, verificada e convertida em estruturas de entidades específicas.

Este módulo contém também o **interpreter**, que processa o ficheiro `input.txt`, onde cada linha corresponde a uma query. O parser comunica diretamente com os módulos de **validação** e **catálogos**, assegurando que apenas dados válidos são carregados em memória.

- **Validação (`valida`)**

Implementa as funções que verificam a correção sintática e lógica dos dados — como o formato das datas, códigos IATA, emails e integridade das reservas.

Cada tipo de entidade possui regras de validação próprias, garantindo a consistência do dataset antes de ser inserido nos catálogos.

- **Gestor de Catálogos (`manager_c`)**

Atua como o elo central entre os diferentes catálogos (voos, passageiros, aeroportos, reservas, aeronaves). Permite aceder e manipular os dados de forma unificada e eficiente.

Este módulo simplifica a execução das queries, ao abstrair o acesso direto às estruturas internas de cada catálogo.

- **Entidades (`entidades`)**

Define as estruturas de dados que representam cada tipo de objeto — como `struct Flight`, `struct Passenger`, `struct Reservation`, etc.

Inclui as funções de criação, acesso e destruição de cada entidade, promovendo o encapsulamento e o reuso de código.

- **Catálogos (`catalogos`)**

Conjunto de estruturas de dados (principalmente *hash tables* da biblioteca **GLib**) que armazenam os objetos validados.

Permitem acesso rápido por identificador ($O(1)$ em média), garantindo eficiência e escalabilidade na execução das queries.

- **Queries (`queries`)**

Responsável por processar as consultas definidas no ficheiro `input.txt`.

Cada função de query analisa os parâmetros recebidos, consulta os catálogos necessários, processa a informação e devolve os resultados formatados.

Os resultados são então enviados para o módulo **output** para serem gravados em ficheiros.

- **Output (**output**)**

Gera os ficheiros `commandN_output.txt` correspondentes aos resultados das queries.

Este módulo assegura que o formato dos outputs segue as especificações do enunciado, distinguindo entre o modo normal e o modo formatado (identificado pelo sufixo “F”).

- **Menu e Datas (**menuNdate**)**

Este módulo auxilia na interação com as datas e operações de tempo, oferecendo funções de comparação, cálculo de intervalos e formatação.

É também usado para organizar as opções ou fluxos de execução internos, simulando um pequeno sistema de menu na lógica do programa.

- **Utils (**utils**)**

Inclui funções de suporte genérico, como manipulação de strings, tratamento de erros, medição de tempos de execução e operações auxiliares sobre arrays e ponteiros.

Serve de apoio transversal a praticamente todos os outros módulos.

[Inserir aqui a Figura 2 — Estrutura de Pastas do Projeto]

Esta organização modular garante uma separação clara entre **camada de dados**, **lógica de negócio** e **camada de saída**, promovendo legibilidade, reuso e facilidade de manutenção.

2.2 Justificação da Estrutura

A adoção desta estrutura modular teve como principais objetivos:

- **Facilitar a manutenção e depuração** do código, isolando responsabilidades em módulos independentes.
- **Permitir o desenvolvimento paralelo** entre os elementos do grupo, sem conflitos de integração.
- **Aumentar a legibilidade e escalabilidade**, permitindo adicionar novas queries ou entidades sem alterar o funcionamento dos restantes módulos.
- **Promover o encapsulamento**, limitando o acesso direto aos dados através de funções controladas.

- **Aproveitar a eficiência da GLib**, que fornece estruturas dinâmicas estáveis e com boa performance para volumes grandes de dados.

Esta abordagem garantiu que o código se mantivesse coerente, limpo e facilmente extensível, suportando tanto a complexidade crescente do projeto como a futura introdução de novas funcionalidades.

2.3 Funcionamento Geral

O ciclo de execução do sistema segue uma sequência bem definida:

1. O utilizador (ou o sistema de testes) executa o programa principal, indicando o caminho para os datasets e para o ficheiro de input.
2. O **parser** lê os ficheiros CSV e envia cada linha para o módulo **valida**, que verifica a integridade dos dados.
3. As entidades válidas são criadas e inseridas nos **catálogos**, através do **manager_c**.
4. O **interpreter** lê o ficheiro **input.txt**, linha a linha, identificando a query correspondente e os seus argumentos.
5. O módulo **queries** executa a consulta, recorrendo aos catálogos para recolher e processar a informação necessária.
6. O módulo **output** formata e escreve o resultado em ficheiros **commandN_output.txt**.
7. Por fim, são libertadas todas as estruturas alocadas dinamicamente, garantindo a ausência de *memory leaks*.

Este fluxo garante uma execução organizada e previsível, desde o carregamento dos dados até à geração dos resultados finais.

2.4 Padrões e Tecnologias Utilizados

O desenvolvimento seguiu uma filosofia de **modularidade progressiva**, criando novos módulos à medida que se identificavam conjuntos de funções com responsabilidades distintas.

Essa abordagem favoreceu a manutenção e leitura do código, além de permitir o desenvolvimento paralelo entre os membros do grupo.

Principais tecnologias e ferramentas:

- **Linguagem C (C99)** – base de toda a implementação.
 - **GLib** – para gestão de *hash tables* e *GArrays*.
 - **Valgrind** – para deteção e eliminação de *memory leaks*.
 - **GDB** – para depuração e análise de execução.
 - **Makefile** – para automatização da compilação, com suporte a *Debug* e *Release*.
-

2.5 Configuração e Compilação

O **Makefile** desempenha um papel essencial na automatização e organização do processo de compilação.

Entre as suas principais funcionalidades destacam-se:

- Flags de compilação específicas para a **GLib** (`pkg-config --cflags --libs glib-2.0`).
 - Criação automática das pastas necessárias (`bin/`, `obj/`, `resultados/`).
 - Geração dos executáveis `programa-principal` e `programa-testes`.
 - Modos de compilação **Debug** (com símbolos de depuração) e **Release** (otimizado).
 - Regra `clean` para remover ficheiros temporários e objetos.
-

2.6 Conclusão

A arquitetura definida fornece uma **base sólida e extensível**, garantindo modularidade, eficiência e facilidade de manutenção.

A interação entre os módulos foi desenhada para ser coesa e intuitiva, permitindo que o sistema responda eficazmente aos requisitos funcionais.

A divisão clara das responsabilidades e a utilização de bibliotecas e ferramentas adequadas tornam este projeto uma implementação robusta, preparada para futuras fases de desenvolvimento e otimizações.

3. Discussão

A estrutura modular definida na fase inicial revelou-se essencial para garantir isolamento entre componentes e facilitar a depuração. Esta divisão permitiu identificar rapidamente erros localizados em etapas específicas, como parsing, validação ou gestão de catálogos, reduzindo significativamente o impacto de falhas entre módulos.

A aplicação rigorosa de encapsulamento, através da exposição controlada de funções nos ficheiros `.h`, evitou acessos diretos a estruturas internas, o que se mostrou determinante para prevenir corrupção de memória e para manter a coerência entre catálogos e entidades. Esta abordagem reforçou a robustez do sistema e contribuiu para a sua escalabilidade.

A gestão de memória foi um dos aspetos mais críticos do desenvolvimento. O Valgrind desempenhou um papel central na identificação de *memory leaks*, *double frees* e acessos inválidos, sobretudo durante a inserção e destruição de entidades nos catálogos. Os resultados evidenciaram uma evolução clara, desde fugas de memória significativas até um estado final praticamente estável, com apenas pequenas perdas residuais a serem corrigidas na fase seguinte.

```
==24606== LEAK SUMMARY:
==24606==    definitely lost: 0 bytes in 0 blocks
==24606==    indirectly lost: 0 bytes in 0 blocks
==24606==    possibly lost: 0 bytes in 0 blocks
==24606==    still reachable: 18,804 bytes in 9 blocks
==24606==           suppressed: 0 bytes in 0 blocks
==24606==
==24606== For lists of detected and suppressed errors, rerun with: -s
==24606== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Para avaliação de desempenho, a metodologia seguiu os requisitos do `programa-testes`, recorrendo à execução repetida de cada query em diferentes máquinas do grupo. Este procedimento permitiu obter tempos médios consistentes e analisar variações causadas por diferenças de hardware.

Os resultados demonstraram que as operações com acesso direto através de *hash tables* apresentaram tempos de resposta quase constantes, enquanto queries que exigem iteração completa ou ordenação exibiram maior variabilidade.

```
Q1: 40 de 40 testes ok!  
Q2: 40 de 40 testes ok  
Q3: 40 de 40 testes ok  
Memória utilizada: 353.0MB  
Tempos de execução:  
Q1: 13.197 ms  
Q2: 9.672 ms  
Q3: 1115.972 ms  
Tempo total: 1.139 s
```

No geral, o desempenho mostrou-se proporcional à complexidade de cada query e refletiu diretamente as escolhas estruturais adotadas. A utilização de estruturas da GLib revelou-se uma decisão acertada, proporcionando eficiência e estabilidade em datasets de larga escala.

4. Conclusão

A Fase 1 do projeto **Airline Query Solver** permitiu consolidar competências fundamentais na **programação estruturada em C**, com foco na **modularização**, **gestão de memória** e **validação de dados**.

Os objetivos propostos foram integralmente alcançados:

- Implementação funcional das três queries obrigatórias;
- Estrutura modular clara e escalável;
- Validação completa dos datasets;
- Makefile funcional e robusto;
- Ausência de *memory leaks* (confirmada via Valgrind).

O grupo reconhece que há ainda oportunidades de melhoria, nomeadamente na **otimização das estruturas de dados** e na preparação para o **modo interativo** exigido na Fase 2.

A experiência obtida foi altamente enriquecedora, proporcionando uma compreensão aprofundada do ciclo de vida de um sistema modular e da importância do controlo rigoroso da memória em C.
